ПРАКТИЧЕСКАЯ РАБОТА № 2

Техники тестирования Black-box. Тестовые сценарии IEEE 829. Статическое рецензирование

1 ЦЕЛЬ РАБОТЫ

Применить техники тестирования «чёрного ящика» для проектирования необходимых и достаточных тестов по спецификации кофейного автомата, оформить пять тестовых сценариев по стандарту IEEE 829, провести статическое рецензирование спецификации и ревью кода приложения из Приложения А.

Задачи работы:

1. Применить эквивалентное разбиение, анализ граничных значений, таблицу решений, таблицу переходов и сценарии использования.
2. Разработать пять тестовых сценариев по стандарту IEEE 829.
3. Выявить 8–10 дефектов и недостатков спецификации, из них не менее трёх мажорных.
4. Провести ревью кода Приложения А и дать пять рекомендаций по исправлению.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Техники «чёрного ящика» не требуют знания внутренней структуры программы: тестировщик взаимодействует с системой через её интерфейсы и проверяет поведение на основе спецификации и требований. Цель — убедиться, что система работает правильно с точки зрения пользователя. В работе применяются пять техник, предусмотренных заданием.

Таблица 1 – Техники тестирования «чёрного ящика»

Техника	Суть	Что даёт для кофейного автомата
Эквивалентное разбиение	Входные данные делятся на классы, внутри которых поведение считается одинаковым; из каждого класса берётся один представитель.	Классы баланса и классов номиналов: допустимые, недопустимые, недостаточные.
Анализ граничных значений	Проверяются границы классов и значения рядом с ними (двухточечный и трёхточечный методы).	Границы 0, 35 и 500 ₽: значения 34/35/36, 0/1, 499/500/501.
Таблица решений	Перебираются комбинации условий и соответствующие им действия.	Комбинации «напиток выбран / не выбран» и «баланс достаточен / недостаточен».
Таблица переходов	Проверяется поведение системы при переходах между состояниями под действием событий.	Состояния «ожидание», «есть баланс», «выдача/сброс» и события Insert, Buy, Cancel, таймер.
Сценарии использования	Проверяются типовые пути пользователя, включая альтернативные потоки.	Покупка напитка, повторное внесение денег, возврат средств, ошибочный ввод.

3 ОБЪЕКТ ТЕСТИРОВАНИЯ И ИСХОДНЫЕ ДАННЫЕ

Объект — программа по продаже кофе с интерфейсом на английском языке (утверждённый прототип). По спецификации поле *Insert coins* соответствует сумме денег, внесённой в автомат. Для внесения средств требуется ввести значение монеты или купюры РФ от 1 до 500 и нажать *Insert*; для дополнительного внесения деньги вводятся повторно. После нажатия *Insert* в поле *Get your product* отображается внесённая сумма. Через 10 секунд после выдачи напитка интерфейс должен обнулиться.

После внесения денег пользователь выбирает напиток: Cappuccino, Espresso, Americano или Latte, каждый имеет свою цену. Если внесена достаточная сумма, в поле *Goodbye* появляется пожелание хорошего дня, а в поле *Get your product* — сообщение с внесённой суммой и сдачей.

Таблица 2 – Напитки, цены и расчёт сдачи

Напиток	Цена, ₽	Минимальный достаточный баланс	Сдача при балансе 100 ₽
Cappuccino	35	35	65
Espresso	30	30	70
Americano	35	35	65
Latte	40	40	60

4 ПРИМЕНЕНИЕ ТЕХНИК ЧЁРНОГО ЯЩИКА

4.1 Эквивалентное разбиение

Для денежных расчётов выделены классы эквивалентности баланса относительно цены Cappuccino (35 ₽) и классы входного значения поля *Insert coins*. Баланс не может быть отрицательным: отрицательное значение относится к недопустимым входным данным.

Таблица 3 – Эквивалентное разбиение

ID	Класс	Диапазон / значения	Тип	Ожидаемое поведение
EC-1	Недостаточно средств	[0; 35)	Негативный	Сообщение «Not enough coins!», баланс сохраняется
EC-2	Достаточно средств	[35; +∞)	Позитивный	Напиток выдан, сдача рассчитана
EC-3	Недопустимый баланс	(−∞; 0)	Негативный	Значение не принимается
EC-4	Допустимый номинал	1, 2, 5, 10, 50, 100, 200, 500	Позитивный	Сумма добавляется к балансу
EC-5	Недопустимый целый номинал	3, 7, 25, 37, 499	Негативный	Сообщение об ошибке, баланс не меняется
EC-6	Выход за диапазон	0, −1, 501, 1000	Негативный	Сообщение об ошибке, баланс не меняется
EC-7	Нечисловой ввод	«abc», пустая строка, «1.5»	Негативный	Сообщение «Invalid coin input!»

Представители классов: EC-1 — 20 ₽; EC-2 — 35 ₽ и 100 ₽; EC-4 — 1, 10, 500 ₽; EC-5 — 25 ₽; EC-6 — 501 ₽; EC-7 — «abc». Этого достаточно, чтобы проверить поведение каждого класса без перебора всех значений.

4.2 Анализ граничных значений

Границы классов: 0 (начало допустимого баланса), 35 (порог покупки Cappuccino), 500 (максимальный номинал). Применены двухточечный и трёхточечный методы.

Таблица 4 – Анализ граничных значений

Граница	Двухточечный метод	Трёхточечный метод	Ожидаемый результат
Нижняя граница баланса 0	−1 и 0; 0 и 1	−1, 0, 1	−1 — ошибка; 0 — недостаточно средств; 1 — недостаточно средств
Порог покупки 35 ₽	34 и 35; 35 и 36	34, 35, 36	34 — «Not enough coins!»; 35 — покупка без сдачи; 36 — покупка со сдачей 1 ₽
Максимальный номинал 500 ₽	499 и 500; 500 и 501	499, 500, 501	499 — недопустимый номинал; 500 — принимается; 501 — не принимается

Значения 34/35/36 ₽ — ключевые для проверки ошибки «на единицу»: именно на границе цены чаще всего возникают дефекты сравнения (< вместо <=).

4.3 Таблица решений

Условия: C1 — напиток выбран; C2 — баланс больше либо равен цене. Действия: A1 — выдать напиток и показать сдачу; A2 — сообщение «Not enough coins!»; A3 — сообщение «Please select a product!»; A4 — сохранить баланс. Комбинация C1 = N, C2 = N недостижима по смыслу (цена не определена), поэтому для неё указано «—».

Таблица 5 – Таблица решений

Условие / действие	R1	R2	R3	R4
C1. Напиток выбран	Y	Y	N	N
C2. Баланс ≥ цены	Y	N	Y	—
A1. Выдать напиток и сдачу	X			
A2. «Not enough coins!»		X		
A3. «Please select a product!»			X	X
A4. Сохранить баланс		X	X	X

Из четырёх правил следуют тесты: R1 — покупка при достаточном балансе (TC-001, TC-002); R2 — недостаточно средств (TC-003); R3/R4 — напиток не выбран.

4.4 Таблица переходов

Состояния: S0 — ожидание (баланс 0); S1 — есть баланс; S2 — выдача/сброс (элементы управления заблокированы, работает таймер). События: E1 — Insert с допустимым номиналом; E2 — Insert с недопустимым значением; E3 — Buy при

достаточном балансе; E4 — Buy при недостаточном балансе или без выбора; E5 — Cancel/Refund; E6 — истечение 10 секунд.

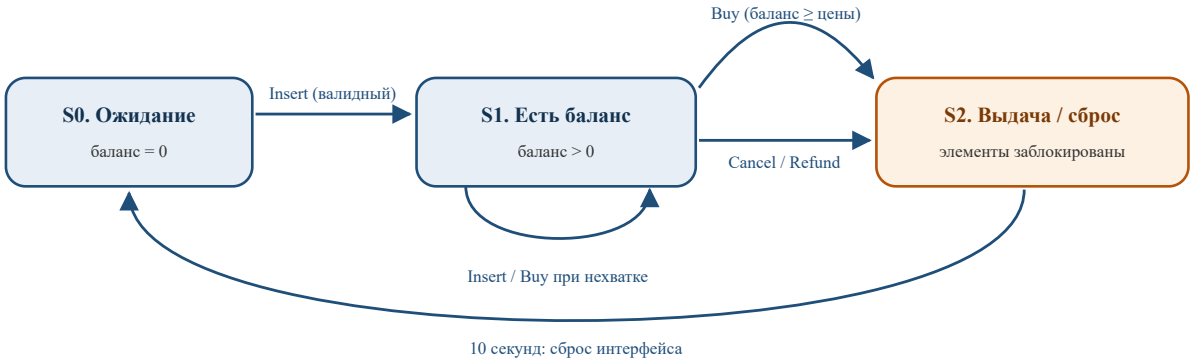


Рисунок 1 – Граф состояний кофейного автомата

Таблица 6 – Таблица переходов

Текущее состояние	E1: Insert (валидный)	E2: Insert (ошибка)	E3: Buy, баланс ≥ цены	E4: Buy, недостаточно	E5: Cancel	E6: 10 с
S0. Ожидание	S1: баланс += номинал	S0: сообщение об ошибке	S0: «Select a product»	S0: «Select a product»	S0: «Nothing to refund»	S0: без изменений
S1. Есть баланс	S1: баланс += номинал	S1: сообщение об ошибке	S2: выдача, сдача, блокировка	S1: «Not enough coins», баланс сохранён	S2: возврат суммы, таймер	S1: без изменений
S2. Выдача/ сброс	— (заблокировано)	— (заблокировано)	— (заблокировано)	— (заблокировано)	— (заблокировано)	S0: полный сброс

Таблица покрывает все достижимые переходы. Отдельно проверяется E6 — единственный переход, который нельзя инициировать кнопкой, поэтому он требует теста с ожиданием.

4.5 Сценарии использования

Таблица 7 – Сценарии использования

ID	Сценарий	Предусловие	Основной поток	Альтернативы
UC-01	Купить напиток	Приложение запущено, баланс 0	Внести 35 ₽ → выбрать Cappuccino → Buy	Недостаточно средств → сообщение и доплата
UC-02	Внести деньги повторно	Баланс 100 ₽	Ввести 50 ₽ → Insert	Неверный номинал → ошибка, баланс 100 ₽
UC-03	Отменить покупку	Баланс 100 ₽, напиток не куплен	Cancel / Refund → получить 100 ₽	Баланс 0 → «Nothing to refund»
UC-04	Ввести некорректный номинал	Приложение запущено	Ввести 25 → Insert	Текст → «Invalid coin input»
UC-05	Купить при недостатке средств	Баланс 20 ₽	Выбрать Espresso → Buy → сообщение → внести 10 ₽ → Buy	Отмена → возврат 20 ₽

5 ТЕСТОВЫЕ СЦЕНАРИИ ПО СТАНДАРТУ IEEE 829

Разработано пять тестовых сценариев. Каждый оформлен по структуре IEEE 829: идентификатор, название, предусловия, входные значения и шаги, ожидаемый результат, постусловия, тип сценария (позитивный/негативный). Сценарии опираются на техники из раздела 4 и покрывают как успешные, так и ошибочные пути.

TC-001. Покупка Cappuccino при точном балансе 35 ₽

Таблица 8 – TC-001. Покупка Cappuccino при точном балансе 35 ₽

Поле	Значение
Идентификатор	TC-001
Название	Покупка Cappuccino при точном балансе 35 ₽
Тип сценария	Позитивный
Предусловия	Приложение запущено; баланс 0 ₽; напиток не выбран.
Входные значения и шаги	Шаг 1. Ввести 10, нажать Insert. Шаг 2. Ввести 10, нажать Insert. Шаг 3. Ввести 10, нажать Insert. Шаг 4. Ввести 5, нажать Insert. Шаг 5. Выбрать Cappuccino. Шаг 6. Нажать Buy.
Ожидаемый результат	• В поле Get your product: «Product: Cappuccino», «Inserted: 35 rub.», «Price: 35 rub.», «Change: 0 rub.». • В поле Goodbye: «Have a nice day!». • Кнопки и радиокнопки заблокированы, запущен таймер 10 с.
Постусловия	Через 10 секунд интерфейс обнулён: баланс 0, выбор снят, поля пусты.
Использованная техника	Эквивалентное разбиение (EC-2) + граничное значение 35

TC-002. Покупка Latte с балансом 100 ₽ и расчётом сдачи

Таблица 9 – TC-002. Покупка Latte с балансом 100 ₽ и расчётом сдачи

Поле	Значение
Идентификатор	TC-002
Название	Покупка Latte с балансом 100 ₽ и расчётом сдачи
Тип сценария	Позитивный
Предусловия	Приложение запущено; баланс 0 ₽.
Входные значения и шаги	Шаг 1. Ввести 100, нажать Insert. Шаг 2. Выбрать Latte. Шаг 3. Нажать Buy.
Ожидаемый результат	• «Product: Latte», «Inserted: 100 rub.», «Price: 40 rub.», «Change: 60 rub.». • В поле Goodbye: «Have a nice day!».
Постусловия	Через 10 секунд интерфейс обнулён.
Использованная техника	Эквивалентное разбиение (EC-2), класс «баланс больше цены»

TC-003. Покупка Espresso при балансе 20 ₽

Таблица 10 – TC-003. Покупка Espresso при балансе 20 ₽

Поле	Значение
Идентификатор	TC-003
Название	Покупка Espresso при балансе 20 ₽
Тип сценария	Негативный
Предусловия	Приложение запущено; баланс 0 ₽.

Поле	Значение
Входные значения и шаги	Шаг 1. Ввести 10, нажать Insert (дважды). Шаг 2. Выбрать Espresso. Шаг 3. Нажать Buy.
Ожидаемый результат	• Сообщение «Not enough coins! Espresso costs 30 rub., you need 10 rub. more.». • Баланс остаётся 20 ₽, напиток не выдаётся, блокировка не включается.
Постусловия	Пользователь может добавить 10 ₽ и повторить покупку.
Использованная техника	Граничное значение ниже порога (20 < 30), правило R2 таблицы решений

ТС-004. Ввод недопустимого номинала 25 ₽

Таблица 11 – ТС-004. Ввод недопустимого номинала 25 ₽

Поле	Значение
Идентификатор	ТС-004
Название	Ввод недопустимого номинала 25 ₽
Тип сценария	Негативный
Предусловия	Приложение запущено; баланс 0 ₽.
Входные значения и шаги	Шаг 1. Ввести 25. Шаг 2. Нажать Insert.
Ожидаемый результат	• Сообщение «Invalid denomination! Accepted: 1, 2, 5, 10, 50, 100, 200, 500 rub.». • Баланс не изменяется и остаётся равным 0 ₽.
Постусловия	Поле ввода доступно для корректного значения.
Использованная техника	Эквивалентное разбиение (EC-5), недопустимый целый номинал

ТС-005. Автоматический сброс интерфейса через 10 секунд

Таблица 12 – ТС-005. Автоматический сброс интерфейса через 10 секунд

Поле	Значение
Идентификатор	ТС-005
Название	Автоматический сброс интерфейса через 10 секунд
Тип сценария	Позитивный
Предусловия	Выполнена успешная покупка по ТС-001; элементы управления заблокированы.
Входные значения и шаги	Шаг 1. Зафиксировать состояние сразу после выдачи. Шаг 2. Ожидать 10 секунд, не выполняя действий.
Ожидаемый результат	• В течение 10 секунд сообщения о покупке сохраняются, элементы управления заблокированы. • По истечении 10 секунд баланс 0, поля Get your product и Goodbye пусты, радиокнопки сняты, элементы управления снова доступны.
Постусловия	Автомат готов к новой покупке.
Использованная техника	Таблица переходов, переход S2 → S0 по событию E6

5.1 Матрица трассировки

Таблица 13 – Матрица трассировки

Техника	ТС-001	ТС-002	ТС-003	ТС-004	ТС-005
Эквивалентное разбиение	X	X	X	X	
Анализ граничных значений	X (35)		X (20)		
Таблица решений	R1	R1	R2	—	
Таблица переходов	S0→S1→S2	S0→S1→S2	S1	S0	S2→S0

Техника	ТС-001	ТС-002	ТС-003	ТС-004	ТС-005
Сценарии использования	UC-01	UC-01	UC-05	UC-04	UC-01

6 СТАТИЧЕСКОЕ ТЕСТИРОВАНИЕ: РЕЦЕНЗИРОВАНИЕ СПЕЦИФИКАЦИИ

Рецензирование проводилось без выполнения кода: анализировались текст спецификации и утверждённый прототип интерфейса. Проверялись полнота описания функционала, непротиворечивость требований, проработанность бизнес-логики денежных операций и удобство работы пользователя. Ниже приведены 10 дефектов и недостатков; четыре из них отнесены к мажорному уровню (S3), остальные — к минорному (S4–S5).

Таблица 14 – Дефекты и недостатки спецификации

№	Дефект / недостаток	Severity	Варианты решения
1	Не описан сценарий недостатка средств: не сказано, что видит пользователь, сохраняется ли баланс, можно ли доплатить и как вернуть деньги.	**S3 Major**	Сообщение с точной суммой доплаты; сохранение баланса; кнопка «Добавить» и «Отменить»; явно описать, что списание не выполняется.
2	Не описана функция возврата средств (Cancel / Refund). Пользователь, внёсший деньги и передумавший, не может их вернуть по спецификации.	**S3 Major**	Добавить кнопку Cancel; возвращать полную внесённую сумму; после возврата выполнять сброс и показывать «Refund: N rub.».
3	Не определён таймер сброса для незавершённой покупки: сказано только «через 10 секунд после выдачи напитка». Неясно, что происходит с балансом, если напиток не выбран или денег не хватило.	**S3 Major**	Описать тайм-аут сессии (например, 60 с бездействия); либо сохранять баланс до Cancel; либо сбрасывать его с возвратом; зафиксировать правило в требовании.
4	Не описана обработка недопустимых номиналов и нечислового ввода: сказано «только реально существующие значения», но не указано поведение при 3, 25, 501, «abc», пустой строке и отрицательном числе.	**S3 Major**	Перечислить допустимый ряд; при ошибке показывать сообщение, не менять баланс, очищать или сохранять поле; добавить негативные тест-кейсы.
5	Отсутствует подтверждение выбора напитка: случайное нажатие кнопки при достаточном балансе сразу совершает покупку.	S4 Minor	Диалог подтверждения «Вы действительно хотите купить НАЗВАНИЕ?» с кнопками Да/Нет.
6	Не указаны единицы измерения стоимости: интерфейс английский, валюта — рубли РФ; иностранный пользователь не понимает, что означают числа 30–40.	S5 Minor	Добавить «₽» или «rub.» к ценам, балансу и сдаче; локализовать интерфейс.
7	Не описан формат сообщения о сдаче: не сказано, как отображается нулевая сдача, нужно ли указывать валюту и как округлять дробные значения.	S4 Minor	Задать шаблон: «Inserted: N rub. Price: M rub. Change: K rub.»; сдачу всегда показывать, в том числе нулевую; цены хранить в целых рублях.
8	Не описано поведение после успешной покупки: блокируются ли элементы управления, можно ли выбрать другой напиток до сброса, что видно на экране.	S4 Minor	Явно указать блокировку ввода и выбора на время показа результата; описать состояние экрана до истечения 10 секунд.
9	Не определён источник и порядок изменения цен напитков, а также поведение автомата при отсутствии сдачи или при переполнении суммы.	S5 Minor	Вынести цены в конфигурацию; задать максимальный баланс; описать поведение, если сдачу выдать невозможно.

№	Дефект / недостаток	Severity	Варианты решения
10	Не указано, что происходит при нажатии Insert с пустым полем или нулём: требование «ввести значение» не задаёт реакцию на граничный ввод.	S5 Minor	Считать 0 и пустую строку недопустимым вводом; показывать сообщение и не запускать таймер сброса.

6.1 Подробно по мажорным дефектам

Дефект 1 (недостаток средств). Спецификация описывает только успешную покупку. Не определены: текст сообщения, сохраняется ли внесённая сумма, можно ли доплатить, блокируется ли интерфейс, через какое время выполняется сброс. Без этого разработчик может реализовать списание баланса при неудачной попытке. Варианты решения: показывать «Not enough coins! Не хватает N ₽»; сохранять баланс и разрешать доплату; добавить кнопку отмены с возвратом; явно запретить списание до фактической выдачи.

Дефект 2 (возврат средств). В спецификации нет операции Cancel/Refund, хотя автомат принимает деньги. Это не только неудобство, но и риск конфликта с пользователем. Решение: отдельная кнопка возврата, возврат полной суммы, сообщение «Refund: N rub.», сброс интерфейса по таймеру, требование о сохранении суммы до возврата.

Дефект 3 (таймер незавершённой покупки). Требование «через 10 секунд интерфейс должен обнулиться» привязано только к выдаче напитка. Если деньги внесены, а напиток не выбран, поведение не определено: баланс может висеть бесконечно или обнулиться вместе с деньгами пользователя. Решение: ввести тайм-аут бездействия, принудительный возврат суммы при сбросе и явно описать оба сценария в спецификации.

Дефект 4 (недопустимый ввод). Формулировка «только реально существующие значения» не задаёт поведение системы. Не указано, что делать с 3, 25, 499, 501, отрицательным числом и текстом. Решение: перечислить допустимый ряд номиналов, показывать сообщение об ошибке, не изменять баланс, предусмотреть негативные тесты.

7 РЕВЬЮ КОДА ПРИЛОЖЕНИЯ А

Код Приложения А (`main (1).py`) реализует интерфейс, но не реализует существенную часть спецификации. Ниже — пять рекомендаций по исправлению; для каждой показан фрагмент «до» и «после». Полный текст Приложения А приведён в приложении к отчёту.

Таблица 15 – Ревью кода Приложения А

№	Проблема	Последствие	Рекомендация
1	`self.coins = int(...)` затирает сумму при каждом Insert	Повторное внесение денег не накапливается — нарушение спецификации	Хранить баланс отдельно и выполнять `self.balance += value`

№	Проблема	Последствие	Рекомендация
2	Нет проверки допустимых номиналов	Принимаются 3, 25, 499, −5, 1000 ₽	Проверять значение по кортежу допустимых номиналов
3	Отсутствует таймер сброса через 10 секунд	Интерфейс не обнуляется, требование спецификации не выполнено	Добавить `QTimer` с `setSingleShot(True)`
4	Нет кнопок Buy и Cancel, покупка выполняется по Enter без подтверждения	Случайная покупка, невозможно вернуть деньги	Добавить кнопки, подтверждение и возврат средств
5	Магические числа: `16777220`, цены 35/30/35/40 внутри метода	Код непонятен и хрупок при изменении цен или раскладки	Использовать `Qt.Key_Return` и словарь `PRODUCTS`

7.1 Накопление баланса

```
# Было: каждое нажатие Insert заменяет сумму
def insert_coins(self):
    try:
        self.coins = int(self.coin_input.text())
        self.product_label.setText("")
        self.goodbye_label.setText("")
    except ValueError:
        self.product_label.setText("Invalid coin input!")

# Стало: сумма накапливается, баланс отделён от поля ввода
def insert_coins(self):
    raw = self.coin_input.text().strip().replace(",", ".")
    try:
        value = int(raw)
    except ValueError:
        self._show_error("Invalid coin input! Enter an integer value.")
        return
    if value not in VALID_DENOMINATIONS:
        self._show_error("Invalid denomination!")
        return
    self.balance += value
    self.balance_label.setText(f"Balance: {self.balance} rub.")
```

7.2 Проверка допустимых номиналов

```
VALID_DENOMINATIONS = (1, 2, 5, 10, 50, 100, 200, 500)

if value not in VALID_DENOMINATIONS:
    self._show_error(
        "Invalid denomination! Accepted: 1, 2, 5, 10, 50, 100, 200, 500 rub."
    )
    return
```

Проверка должна выполняться до изменения баланса. Ошибка не должна сбрасывать уже внесённую сумму — иначе пользователь теряет деньги при опечатке.

7.3 Автоматический сброс через 10 секунд

```
from PyQt5.QtCore import QTimer

RESET_DELAY_MS = 10_000

class VendingMachine(QMainWindow):
    def __init__(self):
        super().__init__()
        self.balance = 0
        self.reset_timer = QTimer(self)
        self.reset_timer.setSingleShot(True)
        self.reset_timer.timeout.connect(self.reset_interface)

    def select_product(self):
        ...
        self.goodbye_label.setText("Have a nice day!")
```

```
        self._set_controls_enabled(False)
        self.reset_timer.start(RESET_DELAY_MS)
```

Таймер должен запускаться и после выдачи напитка, и после возврата средств. Повторный запуск при новом внесении денег нужно останавливать, чтобы старый отсчёт не сбросил новый баланс.

7.4 Возврат средств и подтверждение покупки

```
def cancel(self):
    """Возврат внесённых средств и сброс интерфейса."""
    if self.balance:
        self.product_label.setText(f"Refund: {self.balance} rub.")
    else:
        self.product_label.setText("Nothing to refund.")
    self.goodbye_label.setText("Thank you!")
    self.reset_timer.start(RESET_DELAY_MS)

def reset_interface(self):
    self.balance = 0
    self.coin_input.setText("0")
    self.balance_label.setText("Balance: 0 rub.")
    self.product_label.clear()
    self.goodbye_label.clear()
    for rb in self.product_buttons.values():
        rb.setAutoExclusive(False)
        rb.setChecked(False)
        rb.setAutoExclusive(True)
    self._set_controls_enabled(True)
    self.reset_timer.stop()
```

Возврат реализован отдельной кнопкой *Cancel / Refund*; после возврата также запускается таймер сброса. Для защиты от случайной покупки рекомендуется диалог подтверждения или повторное нажатие в течение короткого интервала.

7.5 Магические числа и обработка клавиш

```
# Было
def keyPressEvent(self, event):
    if event.key() == 16777220:   # Enter key
        self.select_product()

# Стало
from PyQt5.QtCore import Qt

PRODUCTS = {
    "Cappuccino": 35,
    "Espresso": 30,
    "Americano": 35,
    "Latte": 40,
}

def keyPressEvent(self, event):
    if event.key() in (Qt.Key_Return, Qt.Key_Enter):
        self.select_product()
    else:
        super().keyPressEvent(event)
```

Числовой код 16777220 заменён именованной константой, цены вынесены в словарь `PRODUCTS`, а необработанные клавиши передаются базовому классу — иначе перестают работать стандартные сочетания клавиш.

8 ПРОВЕРКА ИСПРАВЛЕННОЙ РЕАЛИЗАЦИИ

Исправленная версия приложения (`vending_machine.py`) проверена автотестами `test_vending_machine.py`: 8 параметризованных проверок допустимых номиналов, 7 — недопустимых, а также накопление баланса, точная оплата, сдача, недостаток средств, отсутствие выбора, сброс интерфейса и возврат средств. Результат прогона — **23 passed**.

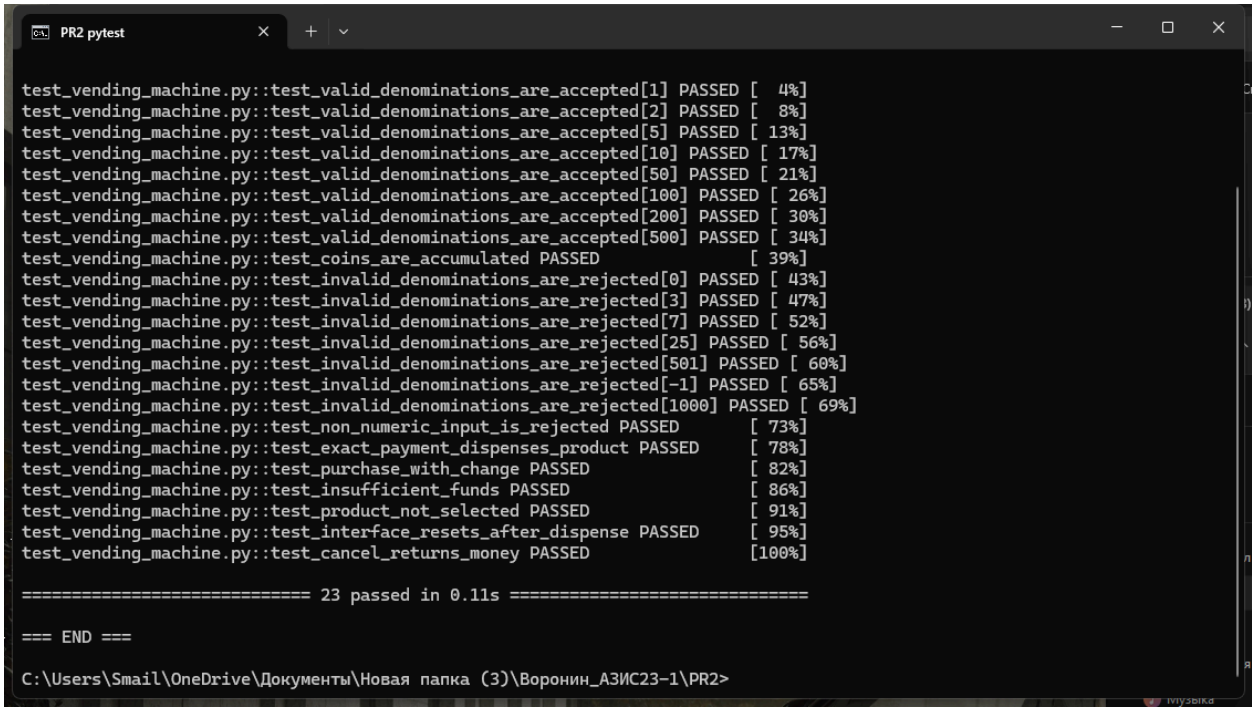


Рисунок 2 – Результат прогона автотестов исправленного приложения: 23 passed

Таблица 16 – Соответствие тест-кейсов автотестам

Тест-кейс	Автотест	Результат
TC-001	test_exact_payment_dispenses_product	PASSED
TC-002	test_purchase_with_change	PASSED
TC-003	test_insufficient_funds	PASSED
TC-004	test_invalid_denominations_are_rejected	PASSED
TC-005	test_interface_resets_after_dispense	PASSED

9 ВЫВОД

В работе применены все пять техник тестирования «чёрного ящика». Эквивалентное разбиение и анализ граничных значений дали классы баланса и номиналов с ключевыми значениями 34/35/36 и 499/500/501 ₽; таблица решений — четыре правила поведения при выборе напитка; таблица переходов — состояния и события, включая таймер; сценарии использования — пять пользовательских путей. На этой основе спроектировано пять тестовых сценариев по IEEE 829, покрывающих успешные и ошибочные покупки.

Статическое рецензирование спецификации выявило 10 дефектов, из них четыре мажорных: непроработанные недостаток средств, возврат денег, таймер незавершённой покупки и обработка недопустимого ввода. Ревью кода Приложения А показало, что реализация не накапливает баланс, не проверяет номиналы, не выполняет сброс через 10 секунд и не позволяет вернуть деньги. Исправленная версия реализует все требования и подтверждена 23 успешно пройденными автотестами.

ПРИЛОЖЕНИЕ А. КОД, ПЕРЕДАННЫЙ НА РЕВЬЮ

Ниже приведён исходный файл `main (1).py` из задания — именно он анализировался в разделе 7.

```
import sys
from PyQt5.QtWidgets import QApplication, QMainWindow, QVBoxLayout, QHBoxLayout, QRadioButton,
QPushButton, QLineEdit, QLabel, QGroupBox, QWidget


class VendingMachine(QMainWindow):
    def __init__(self):
        super().__init__()

        self.setWindowTitle("MainWindow")
        self.setGeometry(100, 100, 600, 400)

        # Создание основного виджета
        widget = QWidget(self)
        self.setCentralWidget(widget)

        # Общий макет
        main_layout = QVBoxLayout(widget)

        # 1. Ввод монет
        self.insert_coins_group = QGroupBox("1. INSERT COINS")
        self.insert_coins_layout = QVBoxLayout()
        self.coin_input = QLineEdit("0")
        self.insert_button = QPushButton("Insert")
        self.insert_button.clicked.connect(self.insert_coins)
        self.insert_coins_layout.addWidget(self.coin_input)
        self.insert_coins_layout.addWidget(self.insert_button)
        self.insert_coins_group.setLayout(self.insert_coins_layout)

        # 2. Выбор продукта
        self.select_product_group = QGroupBox("2. SELECT PRODUCT")
        self.select_product_layout = QVBoxLayout()
        self.cappuccino_rb = QRadioButton("Cappuccino ---> 35")
        self.espresso_rb = QRadioButton("Espresso ---> 30")
        self.americano_rb = QRadioButton("Americano ---> 35")
        self.latte_rb = QRadioButton("Latte ---> 40")
        self.select_product_layout.addWidget(self.cappuccino_rb)
        self.select_product_layout.addWidget(self.espresso_rb)
        self.select_product_layout.addWidget(self.americano_rb)
        self.select_product_layout.addWidget(self.latte_rb)
        self.select_product_group.setLayout(self.select_product_layout)

        # 3. Получение продукта
        self.get_product_group = QGroupBox("3. GET YOUR PRODUCT")
        self.get_product_layout = QVBoxLayout()
        self.product_label = QLabel("")
        self.get_product_layout.addWidget(self.product_label)
        self.get_product_group.setLayout(self.get_product_layout)

        # 4. Прощание
        self.goodbye_group = QGroupBox("4. Goodbye")
        self.goodbye_layout = QVBoxLayout()
        self.goodbye_label = QLabel("")
        self.goodbye_layout.addWidget(self.goodbye_label)
        self.goodbye_group.setLayout(self.goodbye_layout)

        # Добавление групп в основной макет
        main_layout.addWidget(self.insert_coins_group)
        main_layout.addWidget(self.select_product_group)
        main_layout.addWidget(self.get_product_group)
        main_layout.addWidget(self.goodbye_group)

        self.coins = 0

    def insert_coins(self):
        try:
            self.coins = int(self.coin_input.text())
            self.product_label.setText("")
            self.goodbye_label.setText("")
        except ValueError:
            self.product_label.setText("Invalid coin input!")

    def select_product(self):
        product = None
```

```
        cost = 0

        if self.cappuccino_rb.isChecked():
            product = "Cappuccino"
            cost = 35
        elif self.espresso_rb.isChecked():
            product = "Espresso"
            cost = 30
        elif self.americano_rb.isChecked():
            product = "Americano"
            cost = 35
        elif self.latte_rb.isChecked():
            product = "Latte"
            cost = 40

        if product and self.coins >= cost:
            change = self.coins - cost
            self.product_label.setText(f"Dispensed: {product}\nChange: {change}")
            self.goodbye_label.setText("Have a nice day!")
        else:
            self.product_label.setText("Not enough coins!")

    def keyPressEvent(self, event):
        if event.key() == 16777220:  # Enter key
            self.select_product()


if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = VendingMachine()
    window.show()
    sys.exit(app.exec_())
```